

JAVA WITH DSA-DAY 5

HACKATHON

BANK MANAGEMENT SYSTEM:

```
import java.util.*;

class Account
{
    private int accountNumber;
    private String holderName;
    private String password;
    private double balance;

    public Account(int accountNumber, String holderName, String password, double balance)
    {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.password = password;
        this.balance = balance;
    }

    public int getAccountNumber()
    {
        return accountNumber;
    }

    public String getHolderName()
    {
        return holderName;
    }

    public String getPassword()
    {
        return password;
    }

    public double getBalance()
    {
        return balance;
    }
}
```

```

    }
    public void deposit(double amount)
    {
        balance = balance + amount;
    }
    public boolean withdraw(double amount)
    {
        if (amount <= balance)
        {
            balance = balance - amount;
            return true;
        }
        return false;
    }
    public void displayAccount()
    {
        System.out.println("Account Number : " + accountNumber);
        System.out.println("Holder Name   : " + holderName);
        System.out.println("Balance      : " + balance);
    }
}

class Bank
{
    private ArrayList<Account> accounts;
    private int nextAccountNumber;
    public Bank()
    {
        accounts = new ArrayList<>();
        nextAccountNumber = 1001;
    }
    public void createAccount(String holderName, String password, double initialDeposit)
    {

```

```

    if (holderName.trim().isEmpty())
    {
        System.out.println("Holder name cannot be empty.");
        return;
    }
    if (initialDeposit < 0)
    {
        System.out.println("Initial Deposit cannot be negative.");
        return;
    }
    Account account = new Account(nextAccountNumber, holderName, password, initialDeposit);
    accounts.add(account);
    System.out.println("\nAccount Created Successfully.");
    System.out.println("Your Account Number : " + nextAccountNumber);
    nextAccountNumber++;
}

public Account login(String holderName, String password)
{
    for (Account acc : accounts)
    {
        if (acc.getHolderName().equals(holderName) && acc.getPassword().equals(password))
        {
            return acc;
        }
    }
    return null;
}

private Account findAccount(int accountNumber)
{
    for (Account acc : accounts)
    {
        if (acc.getAccountNumber() == accountNumber)

```

```

        {
            return acc;
        }
    }
    return null;
}

public void deposit(int accountNumber, double amount)
{
    Account acc = findAccount(accountNumber);
    if (acc == null)
    {
        System.out.println("Account Not Found.");
        return;
    }
    if (amount <= 0)
    {
        System.out.println("Invalid Amount.");
        return;
    }
    acc.deposit(amount);
    System.out.println("Deposit Successful.");
    System.out.println("Current Balance : " + acc.getBalance());
}

public void withdraw(int accountNumber, double amount)
{
    Account acc = findAccount(accountNumber);
    if (acc == null)
    {
        System.out.println("Account Not Found.");
        return;
    }
    if (amount <= 0)

```

```
        {
            System.out.println("Invalid Amount.");
            return;
        }
        if (acc.withdraw(amount))
        {
            System.out.println("Withdrawal Successful.");
            System.out.println("Current Balance : " + acc.getBalance());
        }
        else
        {
            System.out.println("Insufficient Balance.");
        }
    }
    public void transfer(int fromAccNo, int toAccNo, double amount)
    {
        if (fromAccNo == toAccNo)
        {
            System.out.println("Cannot transfer to the same account.");
            return;
        }
        Account sender = findAccount(fromAccNo);
        Account receiver = findAccount(toAccNo);
        if (sender == null)
        {
            System.out.println("Source Account Not Found.");
            return;
        }
        if (receiver == null)
        {
            System.out.println("Destination Account Not Found.");
            return;
        }
    }
}
```

```

    }
    if (amount <= 0)
    {
        System.out.println("Invalid Amount.");
        return;
    }
    if (sender.withdraw(amount))
    {
        receiver.deposit(amount);
        System.out.println("Transfer Successful.");
        System.out.println("Remaining Balance : " + sender.getBalance());
    }
    else
    {
        System.out.println("Insufficient Balance.");
    }
}

public void searchAccount(int accountNumber)
{
    Account acc = findAccount(accountNumber);
    if (acc == null)
    {
        System.out.println("Account Not Found.");
    }
    else
    {
        acc.displayAccount();
    }
}

public void viewBalance(Account acc)
{
    System.out.println("Current Balance : " + acc.getBalance());
}

```

```

    }
    public void viewMyAccount(Account acc)
    {
        acc.displayAccount();
    }
    public void displayTotalAccounts()
    {
        System.out.println("Total Accounts : " + accounts.size());
    }
    public void displayAllAccounts()
    {
        if (accounts.isEmpty())
        {
            System.out.println("No Accounts Available.");
            return;
        }
        System.out.println("\nALL ACCOUNTS");
        for (Account acc : accounts)
        {
            acc.displayAccount();
            System.out.println();
        }
    }
}

public class Main
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        Bank bank = new Bank();
        while (true)
        {

```

```
System.out.println("\nBANK MANAGEMENT SYSTEM");
System.out.println("1. Create Account");
System.out.println("2. User Login");
System.out.println("3. Admin Login");
System.out.println("4. Exit");
System.out.print("Enter Your Choice : ");
int choice = sc.nextInt();
sc.nextLine();
switch (choice)
{
    case 1:
        System.out.print("Enter Holder Name : ");
        String name = sc.nextLine();
        System.out.print("Create Password : ");
        String password = sc.nextLine();
        System.out.print("Enter Initial Deposit : ");
        double deposit = sc.nextDouble();
        bank.createAccount(name, password, deposit);
        break;
    case 2:
        System.out.print("Enter Holder Name : ");
        String userName = sc.nextLine();
        System.out.print("Enter Password : ");
        String userPassword = sc.nextLine();
        Account user = bank.login(userName, userPassword);
        if (user == null)
        {
            System.out.println("Invalid Login.");
            break;
        }
        System.out.println("Login Successful.");
        int userChoice;
```



```

do
{
    System.out.println("\nUSER MENU");
    System.out.println("1. Deposit");
    System.out.println("2. Withdraw");
    System.out.println("3. Transfer");
    System.out.println("4. View Balance");
    System.out.println("5. View My Account");
    System.out.println("6. Logout");
    System.out.print("Enter Choice : ");
    userChoice = sc.nextInt();
    switch (userChoice)
    {
        case 1:
            System.out.print("Enter Deposit Amount : ");
            double dep = sc.nextDouble();
            bank.deposit(user.getAccountNumber(), dep);
            break;
        case 2:
            System.out.print("Enter Withdrawal Amount : ");
            double wd = sc.nextDouble();
            bank.withdraw(user.getAccountNumber(), wd);
            break;
        case 3:
            System.out.print("Enter Receiver Account Number : ");
            int receiver = sc.nextInt();
            System.out.print("Enter Amount : ");
            double amount = sc.nextDouble();
            bank.transfer(user.getAccountNumber(), receiver, amount);
            break;
        case 4:
            bank.viewBalance(user);

```



```

adminChoice = sc.nextInt();
switch (adminChoice)
{
    case 1:
        System.out.print("Enter Account Number : ");
        int accNo = sc.nextInt();
        bank.searchAccount(accNo);
        break;
    case 2:
        bank.displayAllAccounts();
        break;
    case 3:
        bank.displayTotalAccounts();
        break;
    case 4:
        System.out.println("Admin Logged Out.");
        break;
    default:
        System.out.println("Invalid Choice.");
}
} while (adminChoice != 4);
}
else
{
    System.out.println("Invalid Admin Login.");
}
break;
case 4:
    System.out.println("Thank You Visit Again!");
    sc.close();
    System.exit(0);
default:

```

```
        System.out.println("Invalid Choice.");  
    }  
}  
}  
}
```